

Data Catalog MCP: Servidor MCP de Catálogo de Datos con Validación de Calidad

Redes de Computadoras - CC3067

José Antonio Mérida Castejón

8 de septiembre de 2026

1. Especificación del servidor MCP

1.1. Caso de uso

El servidor implementa un catálogo de datos interno con validación de calidad, dirigido al equipo de analítica de una empresa mediana. El problema que atiende es común en organizaciones que acumulan datasets de distintas fuentes (exportaciones de CRM, ventas, recursos humanos) sin documentación centralizada: los analistas invierten tiempo considerable en determinar qué datos existen, qué significa cada columna, en qué unidades está expresada, y si un extracto recibido cumple las condiciones mínimas de calidad antes de usarlo. El servidor expone herramientas de solo lectura y sin estado que el chatbot anfitrión coordina en lenguaje natural; el LLM coordina las llamadas e interpreta los resultados, pero no genera información por su cuenta.

El escenario concreto se enmarca como Meridian Holdings (inventada por Claude), una empresa ficticia con cuatro unidades de negocio (Meridian Mobile, Meridian Retail, Meridian Commerce y Recursos Humanos corporativos), cada una con su propio sistema de origen. El patrón corresponde a herramientas de uso industrial ya establecido, como los archivos `schema.yml` de dbt, las suites de expectativas de Great Expectations, o catálogos de datos como DataHub o Amundsen, aunque sin constituir un *feature store* propiamente, ya que no calcula ni sirve valores de *features* para inferencia.

1.2. Origen de la información

El servidor obtiene su información de dos fuentes complementarias, ambas locales al servidor y sin dependencias externas en tiempo de consulta:

1. Archivos de datos en formato Parquet (`data/*.parquet`), sobre los cuales se calculan estadísticas y validaciones. Son cinco datasets: suscriptores de Meridian Mobile, ventas de Meridian Retail, transacciones de comercio electrónico de Meridian Commerce, rotación de personal de RR.HH., y un extracto mensual sin validar (copia de suscriptores con inconsistencias introducidas de forma controlada y reproducible, usado para demostrar `validate_dataset`).
2. Un catálogo declarativo en un archivo YAML (`catalog.yaml`), escrito manualmente, que funciona a la vez como diccionario de datos y definición de reglas de calidad. Para cada

columna se declara su tipo, descripción, unidad y las reglas que debe cumplir:

```
- name: customerID
  type: string
  description: Unique customer identifier
  unit: null
  rules:
    - not_null
    - unique
    - regex: '^\\d{4}-[A-Z]{5}$'
      severity: error
```

Dentro de este proyecto, se utilizó Claude para generar los archivos `.yaml` y documentar los datasets. Las reglas de calidad operan a nivel de columna (`not_null`, `unique`, `range`, `allowed_values`, `regex`, `type_check`) o a nivel de dataset (`row_count`, `freshness`). Cada regla declara una severidad: `error` invalida el dataset para uso posterior; `warning` señala una anomalía que amerita revisión sin bloquear el flujo de trabajo.

1.3. Arquitectura interna

El servidor (`cmd/server`) está escrito en Go, todo el framing de mensajes y el manejo del protocolo se implementó a mano. Los paquetes relevantes son:

- `internal/jsonrpc`: framing JSON-RPC 2.0 sobre un `io.Reader/io.Writer` genérico. Lee mensajes delimitados por línea, los decodifica a la estructura `Message` (`jsonrpc`, `id`, `method`, `params`, `result`, `error`), y distingue solicitud de notificación según la presencia del campo `id`.
- `internal/mcp`: implementa el ciclo de vida MCP sobre esa capa (`initialize`, `notifications/initialized`, `tools/list` y `tools/call`), además del despacho a cada herramienta (un archivo `tool_*.go` por herramienta).
- `internal/catalog`: parseo de `catalog.yaml` a estructuras tipadas.
- `internal/data`: resuelve cada entrada del catálogo al archivo Parquet correspondiente y lo lee.
- `internal/embeddings`: cliente de embeddings con dos proveedores intercambiables (Ollama local, Gemini remoto) para `search_catalog`.
- `internal/config`: configuración vía variables de entorno (`TRANSPORT`, `CATALOG_PATH`, `DATA_DIR`, `PORT`, etc.).

Esta separación permite que la *misma* implementación se ejecute sin modificaciones tanto de forma local (stdio) como remota (HTTP/Cloud Run).

1.4. Herramientas expuestas (endpoints MCP)

Todas las herramientas se invocan mediante el método JSON-RPC `tools/call`, identificadas por su campo `name`, con argumentos en `params.arguments`. Son siete en total, definidas cada una en su propio archivo `internal/mcp/tool_*.go` y registradas en `internal/mcp/registry.go`. La Tabla 1 detalla los parámetros de entrada de cada una, tal como los declara su `inputSchema` (JSON Schema, visible vía `tools/list`).

Herramienta	Parámetro	Tipo	Requerido	Descripción
ping	(sin parámetros)			
list_datasets	(sin parámetros)			
describe_dataset	name	string	sí	Nombre del dataset, tal como lo retorna <code>list_datasets</code> .
profile_column	dataset	string	sí	Nombre del dataset.
	column	string	sí	Nombre de la columna, tal como la retorna <code>describe_dataset</code> .
validate_dataset	dataset	string	sí	Nombre del dataset a validar.
find_undocumented_columns	dataset	string	no	Dataset a revisar; si se omite, se revisan todos los registrados.
search_catalog	query	string	sí	Consulta en lenguaje natural.
	limit	integer	no	Máximo de resultados por índice (por defecto 5).

Cuadro 1: Parámetros de entrada de cada herramienta, según su `inputSchema`.

La Tabla 2 muestra, para cada herramienta, los `arguments` de una llamada real y una forma resumida de lo que retorna dentro de `result.content[0].text`.

Como referencia del mensaje JSON-RPC completo (no solo el campo `params/result` recortado), el Listado 1 muestra la solicitud y respuesta íntegras de la llamada a `profile_column` de la tabla anterior:

Listing 1: Solicitud y respuesta JSON-RPC completas de una llamada a `tools/call` (`profile_column`).

```
// Solicitud (cliente -> servidor)
{"jsonrpc":"2.0","id":"4","method":"tools/call",
 "params":{"name":"profile_column",
  "arguments":{"dataset":"meridian_retail_sales","column":"unit_price"}}}

// Respuesta (servidor -> cliente)
{"jsonrpc":"2.0","id":"4","result":{
 "content":[{"type":"text","text":{"\n  \"type\": \"float\", \n  \"count\": 48213, \n  \"nulls
 \": 0, \n  \"min\": 3.5, \n  \"max\": 289.99, \n  \"mean\": 47.82\n}}],
 "isError":false
}}
```

Un argumento faltante o inválido (por ejemplo, un `dataset` o `column` que no existe en el catálogo) no produce un error de protocolo JSON-RPC, sino un resultado de `tools/call` con `isError:true` y el mensaje de error como texto (ver §1.8).

Herramienta	arguments	result (resumido)
ping	{}	"pong"
list_datasets	{}	[{name,description,origin,row_count,updated_at},...]
describe_dataset	{"name":"meridian_retail_sales"}	{name,columns:[{name,type,description,unit,rules},...]}
profile_column	{"dataset":"meridian_retail_sales","column":"unit_price"}	{type:"float",count:48213,nulls:0,min:3.5,max:289.99,mean:47.82}
validate_dataset	{"dataset":"meridian_mobile_subscribers_raw"}	{rules_evaluated:12,passed:9,failed:3,violations:[...]}
find_undocumented_columns	{"dataset":"meridian_retail_sales"}	[{dataset,columns:["internal_flag"]}]
search_catalog	{"query":"customerchurn","limit":3}	{datasets:[...],columns:[...]}

Cuadro 2: Argumentos de la solicitud y forma resumida del resultado para cada herramienta. El Listado 1 muestra el mensaje JSON-RPC completo, sin resumir, para `profile_column`.

1.5. Búsqueda semántica en dos niveles

`search_catalog` construye al iniciar el servidor dos índices vectoriales, uno de datasets (nombre, descripción, origen) para consultas de granularidad amplia y otro de columnas (nombre y descripción de la columna junto con la descripción de su dataset, ya que el nombre de una columna por sí solo suele ser demasiado breve para una representación vectorial informativa). La recuperación es por similitud coseno mediante multiplicación matricial; dado el tamaño reducido del índice (≈ 100 vectores), una búsqueda exhaustiva es apropiada y no se justifica una base de datos vectorial dedicada.

1.6. Privacidad y servicios externos

La generación de embeddings se delega a un servicio externo (Ollama local o Gemini), lo que evita incorporar un modelo local al contenedor y mantiene una imagen ligera con arranques en frío reducidos. El servidor transmite a ese servicio únicamente metadatos (nombres de dataset, nombres de columna, descripciones y unidades, redactados manualmente en el catálogo) y las consultas del usuario, nunca registros ni valores de los archivos de datos. Esta separación es una decisión de diseño que hace viable el caso de uso en un entorno donde los datos suelen estar sujetos a restricciones contractuales o regulatorias.

1.7. Transporte y despliegue

- `stdio` (local): mensajes JSON delimitados por línea sobre `stdin/stdout` del proceso hijo, spawnado por el anfitrión (`cmd/host` o `cmd/client`) según `client.json`.
- `HTTP` (remoto): un único endpoint (`POST/`) que recibe un objeto JSON-RPC por solicitud y responde con el resultado correspondiente. Se activa con `TRANSPORT=http`; el puerto se toma de la variable `PORT` inyectada por la plataforma.

El servidor remoto se desplegó en Google Cloud Run, construido mediante Cloud Build directamente desde el repositorio de GitHub (sin necesidad de Docker local): una imagen `distroless` de dos

etapas (compilación Go, luego binario estático más `catalog/` y `data/*.parquet`). Las credenciales de embeddings (`EMBEDDINGS_API_KEY`) se inyectan vía Secret Manager, nunca como variable de entorno en texto plano. Ambos transportes exponen exactamente los mismos métodos MCP: `initialize`, `tools/list` y `tools/call`, por lo que el anfitrión usa el servidor remoto de forma idéntica al local, solo cambiando la entrada `url` por `command` en `client.json`.

1.8. Subconjunto de MCP implementado

El proyecto implementa un subconjunto deliberado de la especificación MCP, suficiente para el ciclo de vida completo (*handshake*, descubrimiento de herramientas, invocación) contra cualquier servidor MCP estándar, pero sin las extensiones agregadas en revisiones posteriores del protocolo.

Operaciones soportadas. Ambos lados (cliente en `internal/client`, servidor en `internal/mcp`) implementan únicamente:

- `initialize / notifications/initialized`: *handshake* de sesión, donde se declara `protocolVersion` (2025-11-25) y la única capacidad anunciada, `tools`.
- `tools/list`: descubrimiento de herramientas disponibles, con su `inputSchema` (JSON Schema).
- `tools/call`: invocación de una herramienta. El resultado siempre es un bloque de contenido de texto (`content: [{type: "text", text}]`), con `isError` para distinguir éxito de fallo.

No se implementan `resources`, `prompts`, `sampling`, autorización OAuth/OIDC, *elicitation*, *batching* de JSON-RPC, ni *tasks* de larga duración. La ausencia de estas extensiones no afecta el funcionamiento del *host* contra servidores de terceros, el cliente (`internal/client/stdio.go`) responde `methodnotfound` a cualquier solicitud iniciada por el servidor que no reconozca (p.ej. `sampling/createMessage`), en vez de fallar o bloquearse, precisamente para tolerar servidores más completos sin implementar todas sus capacidades.

Manejo de errores de herramienta. Un argumento inválido o una referencia no resoluble (dataset o columna inexistente) se reporta como resultado de `tools/call` con `isError:true` y el mensaje de error como texto, en vez de un error de protocolo JSON-RPC (-32602). Este es el comportamiento que exige la revisión 2025-11-25 de la especificación, permite que el modelo lea el error como texto y se autocorrija en el siguiente turno en vez de recibir una falla opaca a nivel de RPC. Solo una falla interna genuina del servidor (no atribuible a los argumentos recibidos) se reporta como error JSON-RPC (-32603) y se registra en el log del servidor.

Transporte HTTP. El transporte HTTP implementado expone un único endpoint: `POST/` recibe un mensaje JSON-RPC en el cuerpo y responde con el resultado correspondiente (o 204 si era una notificación). No implementa `GET/Server-Sent Events`, negociación de `Content-Type` vía encabezado `Accept`, ni el encabezado `Mcp-Session-Id`, las tres piezas que la especificación agrega en *Streamable HTTP* (revisión 2025-03-26) para servidores con estado de sesión y mensajes servidor→cliente. Ninguna de las siete herramientas del servidor hace trabajo de larga duración ni necesita empujar datos al cliente entre solicitudes; cada llamada es una consulta puntual sobre el catálogo o los archivos Parquet, que entra y sale en una sola petición-respuesta. Por eso un ciclo simple de `POST` por mensaje es suficiente para este proyecto.

1.9. Arquitectura del proyecto: host, cliente y servidor

El proyecto instancia los tres actores de la arquitectura MCP (§1 del enunciado) de forma explícita y verificable en el código:

Actor	Componente	Responsabilidad
Anfitrión (<i>host</i>)	<code>cmd/host</code>	Proceso Go que lee <code>client.json</code> , instancia un Router (uno o más clientes), coordina el ciclo de llamadas del LLM (<code>internal/agent</code>) y sirve la conversación al frontend vía <i>server-sent events</i> en <code>POST/api/chat</code> .
Cliente	<code>internal/client</code> (Router , <code>stdioClient</code> , <code>httpClient</code>)	Un cliente por servidor configurado; implementa <code>initialize/tools/list/tools/call</code> sobre el transporte correspondiente (<code>stdio</code> o <code>HTTP</code>) y expone una interfaz uniforme (Client) al <i>host</i> , sin importar el transporte real.
Servidor	<code>cmd/server</code> (<code>internal/mcp</code>)	Ejecuta las herramientas del catálogo de datos. El mismo binario corre local (<code>stdio</code>) o remoto (<code>HTTP</code>), sin cambios de lógica; solo cambia <code>TRANSPORT</code> .

Cuadro 3: Los tres actores de la arquitectura MCP, mapeados a los componentes del proyecto.

En este proyecto conviven además dos servidores MCP oficiales de terceros (Filesystem y Git, §1.11 abajo) y el servidor propio, todos coordinados por el mismo *host* a través del mismo **Router**. Es exactamente el escenario de múltiples clientes/servidores del diagrama de arquitectura del enunciado (Cliente MCP 1, Cliente MCP 2, ...).

Compatibilidad con Diferentes Servidores. `Router.NewRouter` (`internal/client/router.go`) no asume nada específico sobre el servidor más allá del subconjunto de §1.8. Por cada entrada de `client.json`, construye un `httpClient` si la entrada declara `url`, o un `stdioClient` (`ConnectWithEnv`, que hace *spawn* de `command/args`) si declara `command`, y llama `Initialize()` sobre cualquiera de los dos de forma idéntica. Esto significa que cualquier servidor MCP que implemente el ciclo `initialize/tools/list/tools/call` (prácticamente todo servidor MCP real), puede agregarse a `client.json` sin modificar una sola línea del *host*. Esto se verificó en la práctica con los servidores oficiales Filesystem y Git (§1.11), ambos de implementaciones ajenas a este proyecto.

Compatibilidad con Diferentes Hosts. Además de usarse desde el *host* de este proyecto, `cmd/server` se agregó como servidor MCP a Claude Code, un anfitrión MCP de terceros completamente independiente de este repositorio, para confirmar que el servidor es interoperable con cualquier anfitrión conforme a MCP y no solo con el *host* propio. Esto valida en la práctica la premisa central del protocolo (§1 del enunciado), que una herramienta implementada una sola vez es utilizable por cualquier anfitrión compatible sin adaptación.

1.10. Ejecución: local vs. remoto

El código del servidor y los métodos MCP expuestos son idénticos en ambos casos (§1.8); lo único que cambia es la entrada `data-catalog` de `client.json` (`command` para local, `url` para remoto, §1.9). Las instrucciones completas de instalación y arranque, tanto local como el despliegue a Google Cloud Run, están documentadas en el `README.md` del repositorio; a continuación se reproducen los comandos esenciales de cada camino.

```

| ./scripts/setup.sh                # genera .env y client.json, compila bin/data-catalog-mcp
| go run ./cmd/host                 # anfitrión + puente de chat, :8090
| cd frontend && bun install && bun dev # interfaz web de chat

```

```

| gcloud run deploy data-catalog-mcp \
|   --source . \
|   --region northamerica-south1 \
|   --allow-unauthenticated \
|   --set-env-vars TRANSPORT=http

```

Cloud Build compila la imagen a partir del `Dockerfile` del repositorio sin necesidad de Docker local. La URL del servicio resultante es lo único que se copia a `client.json`; ningún otro componente del proyecto cambia.

1.11. Servidores MCP oficiales adicionales

Además del servidor propio, el anfitrión se conecta a dos servidores MCP oficiales de Anthropic (requeridos por el enunciado, §3.1 punto 4), consumidos tal cual, sin modificaciones, para demostrar interoperabilidad del cliente contra implementaciones de terceros. Ambos se declaran en `client.json` con transporte stdio (`command/args`), igual que cualquier otro servidor local.

Campo	Valor
Paquete	@modelcontextprotocol/server-filesystem
Transporte	stdio, npx-y@modelcontextprotocol/server-filesystem<root-path>
Alcance	/home/Tono/Documents/data-catalog-mcp
Rol	Consumido sin modificar; demuestra interoperabilidad cliente-servidor de terceros.

Cuadro 4: Servidor MCP oficial: Filesystem.

Herramienta	Propósito
<code>read_file / read_text_file</code>	Lee el contenido de un archivo.
<code>write_file</code>	Crea o sobrescribe un archivo.
<code>edit_file</code>	Aplica una edición estilo diff.
<code>create_directory</code>	Crea un directorio.
<code>list_directory</code>	Lista el contenido de un directorio.
<code>move_file</code>	Mueve o renombra un archivo.
<code>search_files</code>	Búsqueda recursiva por nombre/contenido.
<code>get_file_info</code>	Metadatos (<code>stat</code>) de un archivo.

Cuadro 5: Herramientas representativas expuestas por el servidor Filesystem (según su propio `tools/list`).

Campo	Valor
Paquete	mcp-server-git (Python, módulo mcp_server_git)
Transporte	stdio, python3-mmcp_server_git-r<ruta-repositorio>
Alcance	/home/Tono/Documents/data-catalog-mcp
Rol	Consumido sin modificar.

Cuadro 6: Servidor MCP oficial: Git.

Herramienta	Propósito
git_status	Estado del árbol de trabajo.
git_add	Agrega archivos al stage.
git_reset	Quita del stage todos los cambios agregados.
git_commit	Crea un commit.
git_diff / git_diff_staged / git_diff_unstaged	Muestra diferencias.
git_log	Historial de commits.
git_show	Contenido de un commit específico.
git_branch / git_checkout / git_create_branch	Operaciones de rama.

Cuadro 7: Herramientas expuestas por el servidor Git, según su `tools/list`. No incluye `git_init`.

2. Análisis de la comunicación (Wireshark)

Se realizaron dos capturas complementarias contra el mismo servidor MCP (mismo binario, mismo código, mismos mensajes JSON-RPC), diferenciadas únicamente por el transporte:

- **Captura remota:** el anfitrión conectado al servidor desplegado en Google Cloud Run (`data-catalog-mcp-53268160127.northamerica-south1.run.app`) sobre HTTPS. Filtrada en Wireshark con `ipv6.addr==2600:1900:4243:200::`. Al ser HTTPS, el contenido de aplicación viaja cifrado; esta captura se usa para el análisis de las capas de enlace, red y transporte (§2.2), incluyendo el handshake TCP y TLS reales contra el servidor en la nube.
- **Captura local:** el mismo anfitrión conectado al mismo servidor ejecutándose localmente con `TRANSPORT=http` (sin TLS), filtrada con `tcp.port==8091`. Dado que el formato y contenido de los mensajes JSON-RPC es idéntico independientemente del transporte, esta captura se usa para inspeccionar y clasificar los mensajes JSON-RPC en texto plano (§2.1), lo cual no es posible directamente sobre la captura remota sin descifrar la sesión TLS.

2.1. Mensajes JSON-RPC capturados

Se identifican tres tipos de mensaje JSON-RPC en la especificación: solicitud (`request`, contiene `id` y `method`), respuesta (`response`, contiene `id` y `result` o `error`), y notificación (`notification`, sin `id`, no espera respuesta, usada por MCP para `notifications/initialized`). Los mensajes de sincronización (`initialize / notifications/initialized`) son los que establecen el protocolo antes de que el anfitrión pueda invocar herramientas; el resto son solicitud/respuesta ordinarias de operación.

La Tabla 8 clasifica los mensajes de la captura local (`tcp.streameq10`, misma sesión TCP durante toda la conversación), identificados por su número de frame de Wireshark.

Frame	Tipo	Método	HTTP	Observaciones
24	Solicitud	<code>initialize</code>	200	<code>Content-Length:172</code> ; negocia <code>protocolVersion:"2025-11-25"</code> y declara <code>clientInfo</code> .
26	Respuesta	—	200	<code>Content-Length:154</code> ; responde al <code>id:1</code> con <code>capabilities.tools</code> y <code>serverInfo</code> .
28	Notificación	<code>notifications/initialized</code>	204	<code>Content-Length:54</code> ; sin campo <code>id</code> en el cuerpo. El servidor no emite un objeto JSON-RPC de respuesta (no corresponde a una notificación); a nivel HTTP responde <code>204NoContent</code> sin cuerpo (frame 29).
39	Solicitud	<code>tools/list</code>	200	<code>id:2</code> ; sin parámetros.
40	Respuesta	—	200	Arreglo <code>tools</code> completo, con el <code>inputSchema</code> de las siete herramientas (Tabla 1).
745	Solicitud	<code>tools/call</code>	200	<code>id:4</code> ; <code>params:{"name":"list_datasets","arguments":{}}</code> .
746	Respuesta	—	200	<code>Content-Length:2016</code> ; <code>content[0].text</code> con el arreglo de los cinco datasets registrados.

Cuadro 8: Clasificación de los mensajes JSON-RPC observados en la captura local, por número de frame de Wireshark.

La misma sesión capturó además una segunda ronda de `tools/list` (otra instancia del *host*, mismo puerto) y dos llamadas adicionales, `search_catalog` y `describe_dataset`, generadas por una conversación real a través del frontend en vez de solicitudes manuales. El Listado 2 reproduce, en texto plano tal como aparece en *Follow → HTTP Stream*, el tramo `initialize → notifications/initialized → tools/call (list_datasets)`:

Listing 2: Fragmento de la captura local (`tcp.stream eq 10`), en texto plano.

```

POST / HTTP/1.1
Host: localhost:8091
Content-Length: 172
Content-Type: application/json

{"jsonrpc":"2.0","id":1,"method":"initialize","params":{"protocolVersion":"2025-11-25","capabilities":{},"clientInfo":{"name":"data-catalog-mcp-client","version":"0.0.1"}}}
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 154

{"jsonrpc":"2.0","id":1,"result":{"protocolVersion":"2025-11-25","capabilities":{"tools":{}}, "serverInfo":{"name":"data-catalog-mcp","version":"0.0.1"}}}

POST / HTTP/1.1
Host: localhost:8091

```

```

Content-Length: 54
Content-Type: application/json

{"jsonrpc":"2.0","method":"notifications/initialized"}
HTTP/1.1 204 No Content

POST / HTTP/1.1
Host: localhost:8091
Content-Length: 95
Content-Type: application/json

{"jsonrpc":"2.0","id":4,"method":"tools/call","params":{"arguments":{},"name":"list_datasets"}}
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 2016

{"jsonrpc":"2.0","id":4,"result":{"content":[{"type":"text","text":["\n {\n      \"name\": \"\n      meridian_mobile_subscribers\", \n      ... \n      \"row_count\": 7043 \n    }, \n      ... \n    ]}]}

```

La captura confirma, a nivel de enlace y red, lo mismo que §1.10 argumenta a nivel de arquitectura: origen y destino son ::1 (loopback IPv6) tanto en el SYN inicial como en cada solicitud HTTP posterior (Figura 1) no hay segundo host, por lo que la capa de red no realiza ningún trabajo de enrutamiento efectivo a diferencia de la captura remota (§2.2).

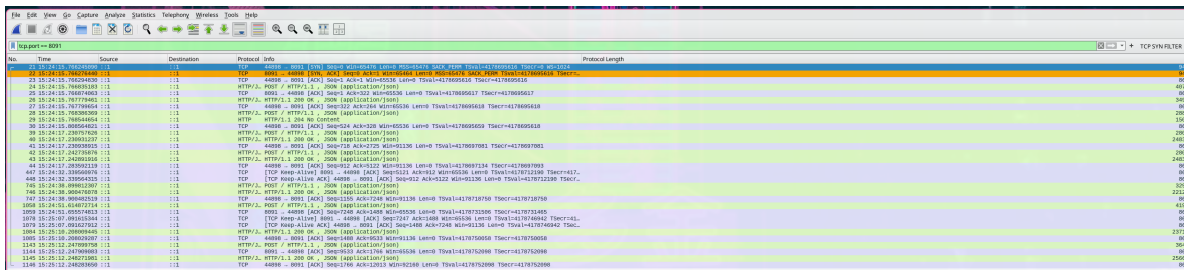


Figura 1: Captura local: sesión completa filtrada con `tcp.port == 8091`, incluyendo el three-way handshake TCP (frames 21–23) y siete pares solicitud/respuesta HTTP sobre ::1.

```

POST / HTTP/1.1
Host: localhost:8091
User-Agent: Go-http-client/1.1
Content-Length: 172
Content-Type: application/json
Accept-Encoding: gzip

{"jsonrpc":"2.0","id":1,"method":"initialize","params":{"protocolVersion":"2025-11-25","capabilities":{},"clientInfo":{"name":"data-catalog-mcp-client","version":"0.0.1"}}}

HTTP/1.1 200 OK
Content-Type: application/json
Date: Tue, 08 Sep 2026 21:24:15 GMT
Content-Length: 154

{"jsonrpc":"2.0","id":1,"result":{"protocolVersion":"2025-11-25","capabilities":{"tools":{},"serverInfo":{"name":"data-catalog-mcp","version":"0.0.1"}}}

POST / HTTP/1.1
Host: localhost:8091
User-Agent: Go-http-client/1.1
Content-Length: 54
Content-Type: application/json
Accept-Encoding: gzip

{"jsonrpc":"2.0","method":"notifications/initialized"}

HTTP/1.1 204 No Content
Date: Tue, 08 Sep 2026 21:24:15 GMT

POST / HTTP/1.1
Host: localhost:8091
User-Agent: Go-http-client/1.1
Content-Length: 46
Content-Type: application/json
Accept-Encoding: gzip

{"jsonrpc":"2.0","id":2,"method":"tools/list"}

HTTP/1.1 200 OK
Content-Type: application/json
Date: Tue, 08 Sep 2026 21:24:17 GMT
Transfer-Encoding: chunked

{"jsonrpc":"2.0","id":2,"result":{"tools":[{"name":"profile_column","description":"Profiles one column of a dataset: declared type, count, null count/ratio, distinct count, numeric min/max/mean, and either a full value-frequency table (low-cardinality columns) or sample values (high-cardinality columns).","inputSchema":{"properties":{"column":{"description":"Column name, as returned by describe_dataset.","type":"string"},"dataset":{"description":"Dataset name, as returned by list_datasets.","type":"string"}}},"required":["dataset","column"],"type":"object"}],"name":"validate_dataset","description":"Validates a dataset's data against its catalog quality rules (not_null, unique, range, allowed_values, regex, type_check, row_count) and reports any violations.","inputSchema":{"properties":{"dataset":{"description":"Dataset name, as returned by list_datasets.","type":"string"},"required":{"dataset"},"type":"object"}],"name":"find_undocumented_columns","description":"Finds Parquet columns not declared in the catalog's data dictionary. Checks one dataset if given, otherwise every registered dataset.","inputSchema":{"properties":{"dataset":{"description":"Dataset name to check. Omit to check every registered dataset.","type":"string"},"type":"object"}],"name":"search_catalog","description":"Searches dataset and column descriptions for the ones most relevant to a query. Uses semantic search when an embeddings service is configured, otherwise keyword matching.","inputSchema":{"properties":{"limit":{"description":"Maximum number of results to return (default 5).","type":"integer"},"query":{"description":"Natural language search query.","type":"string"},"required":{"query"},"type":"object"}],"name":"ping","description":"Health check: returns pong.","inputSchema":{"properties":{"type":"object"}],"name":"list_datasets","description":"Lists the registered datasets with their description, origin, and row count.","inputSchema":{"properties":{"type":"object"}],"name":"describe_dataset","description":"Describes one registered dataset: its metadata, row count, and column dictionary.","inputSchema":{"properties":{"name":{"description":"Dataset name, as returned by list_datasets.","type":"string"},"required":{"name"},"type":"object"}}}]}}

POST / HTTP/1.1
Host: localhost:8091
User-Agent: Go-http-client/1.1
Content-Length: 46
Content-Type: application/json
Accept-Encoding: gzip

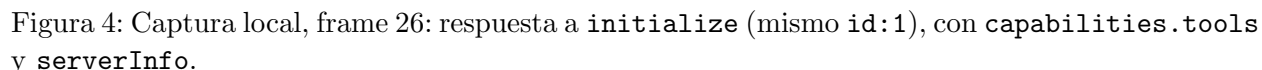
{"jsonrpc":"2.0","id":3,"method":"tools/list"}

HTTP/1.1 200 OK
Content-Type: application/json
Date: Tue, 08 Sep 2026 21:24:17 GMT
Transfer-Encoding: chunked

{"jsonrpc":"2.0","id":3,"result":{"tools":[{"name":"profile_column","description":"Profiles one column of a dataset: declared type, count, null count/ratio, distinct count, numeric min/max/mean, and either a full value-frequency table (low-cardinality columns) or sample values (high-cardinality columns).","inputSchema":{"properties":{"column":{"description":"Column name, as returned by describe_dataset.","type":"string"},"dataset":{"description":"Dataset name, as returned by list_datasets.","type":"string"}}},"required":["dataset","column"],"type":"object"}],"name":"validate_dataset","description":"Validates a dataset's data against its catalog quality rules (not_null, unique, range, allowed_values, regex, type_check, row_count) and reports any violations.","inputSchema":{"properties":{"dataset":{"description":"Dataset name, as returned by list_datasets.","type":"string"},"required":{"dataset"},"type":"object"}],"name":"find_undocumented_columns","description":"Finds Parquet columns not declared in the catalog's data dictionary. Checks one dataset if given, otherwise every registered dataset.","inputSchema":{"properties":{"dataset":{"description":"Dataset name to check. Omit to check every registered dataset.","type":"string"},"type":"object"}],"name":"search_catalog","description":"Searches dataset and column descriptions for the ones most relevant to a query. Uses semantic search when an embeddings service is configured, otherwise keyword matching.","inputSchema":{"properties":{"limit":{"description":"Maximum number of results to return (default 5).","type":"integer"},"query":{"description":"Natural language search query.","type":"string"},"required":{"query"},"type":"object"}],"name":"ping","description":"Health check: returns pong.","inputSchema":{"properties":{"type":"object"}],"name":"list_datasets","description":"Lists the registered datasets with their description, origin, and row count.","inputSchema":{"properties":{"type":"object"}],"name":"describe_dataset","description":"Describes one registered dataset: its metadata, row count, and column dictionary.","inputSchema":{"properties":{"name":{"description":"Dataset name, as returned by list_datasets.","type":"string"},"required":{"name"},"type":"object"}}}]}}

```

Figura 2: Captura local: *Follow* → *HTTP Stream* de `tcp.stream` eq 10, la conversación completa en texto plano (compárese con el Listado 2).



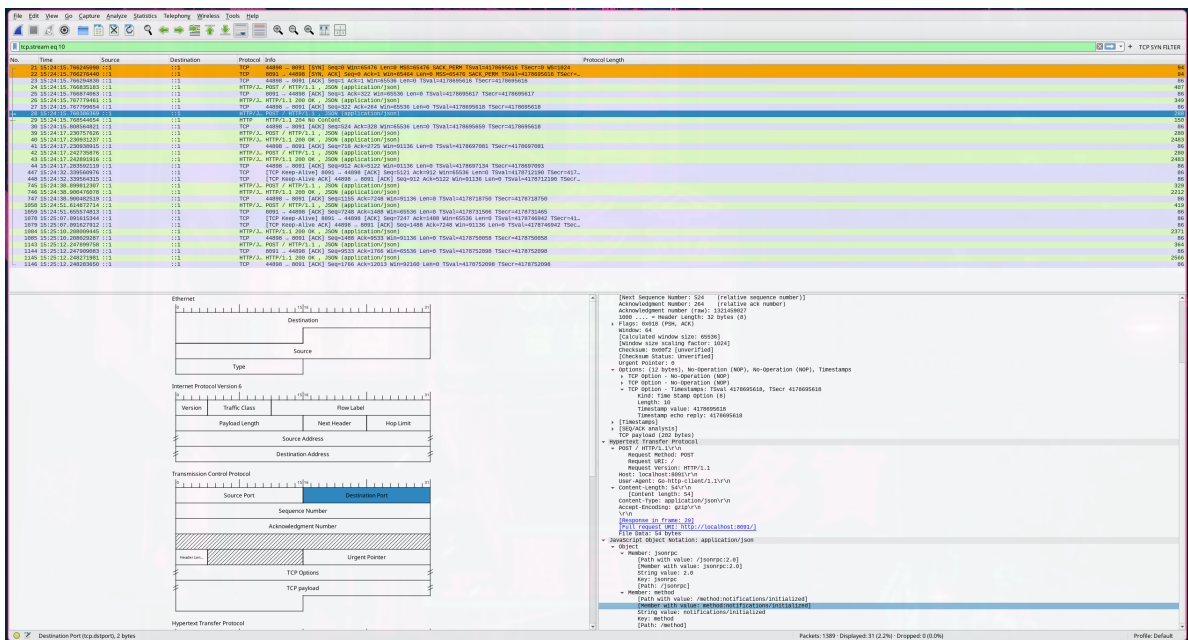


Figura 5: Captura local, frame 28: notifications/initialized. El árbol de disección muestra el objeto JSON completo (jsonrpc, method) sin campo id, lo que la identifica como notificación.

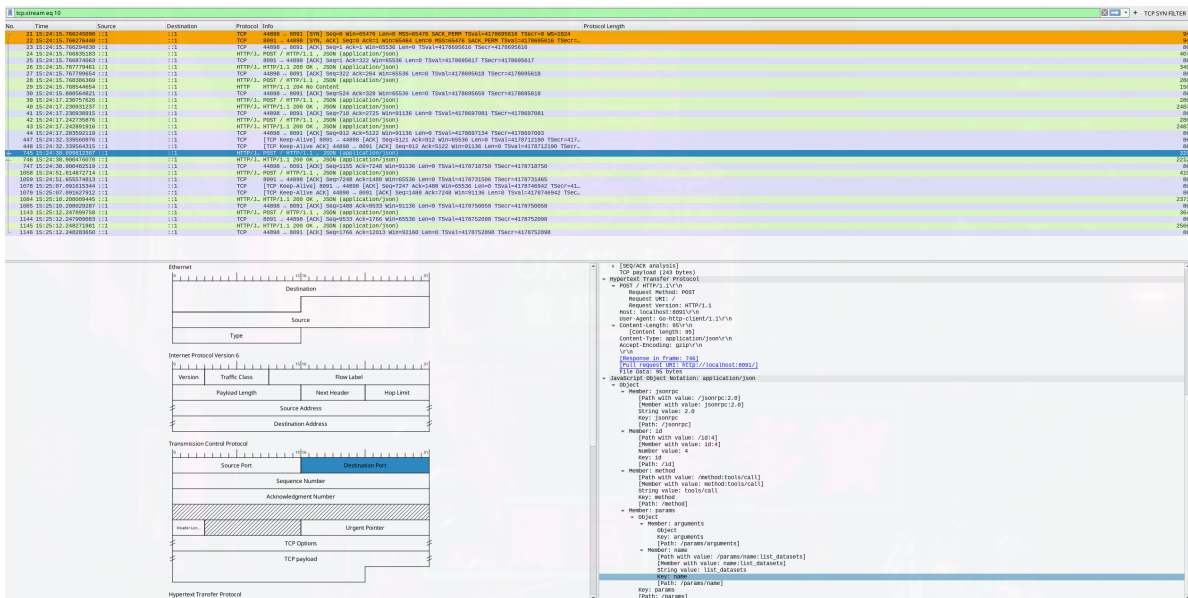


Figura 6: Captura local, frame 745: solicited tools/call para list_datasets (id:4, arguments vacío).

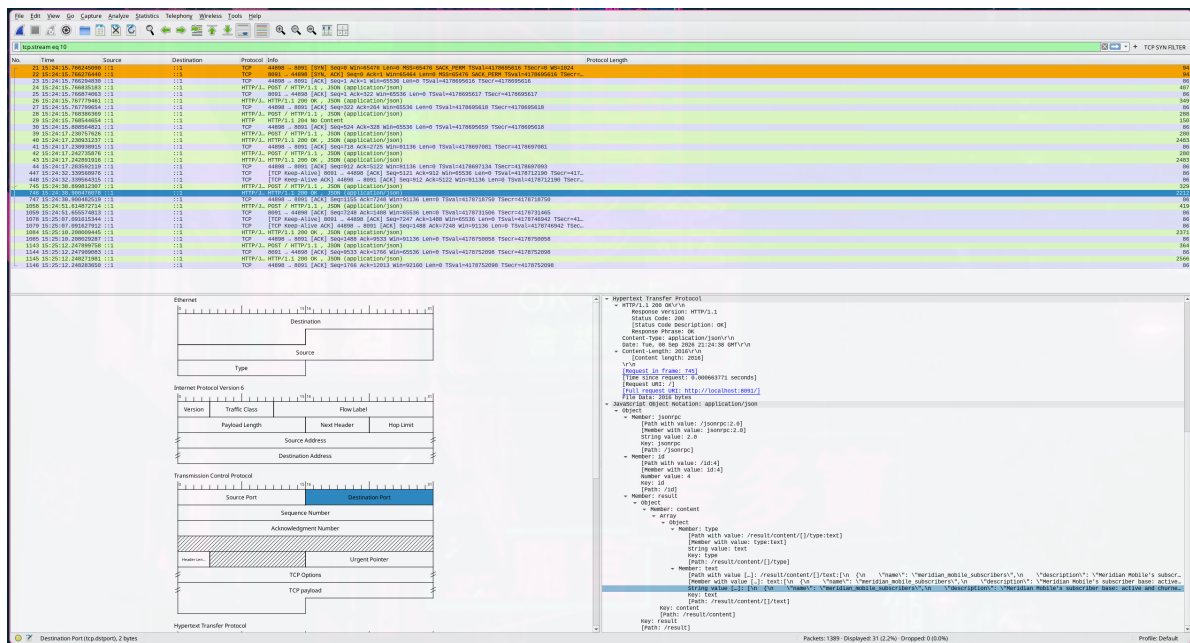


Figura 7: Captura local, frame 746: respuesta a la llamada anterior, `content[0].text` con el arreglo de datasets registrados.

2.2. Análisis por capa

Capa de enlace. En la captura remota (Figura 8), Wireshark reporta un encabezado Ethernet II real: dirección MAC origen 84:01:12:d3:19:e9 (identificada por el fabricante como KaonGroup, la interfaz de red local de la máquina que capturó el tráfico) y dirección MAC destino 0e:62:e7:72:da:65. Esta última no es la MAC del servidor de Cloud Run, ya que por diseño una dirección MAC nunca sobrevive más de un salto, sino la del siguiente dispositivo en la ruta (el gateway/router local). Esta es la única capa donde se opera sobre un segmento físico compartido, a partir de la capa de red la comunicación ya es de extremo a extremo lógico.

Capa de red. El mismo frame (Figura 8) muestra IPv6: dirección origen 2800:98:1124:aa7::2 (la máquina cliente) y destino 2600:1900:4243:200:: (la IP pública de Cloud Run, dentro del bloque de Google Cloud).

Capa de transporte. Se observa el three-way handshake completo de TCP (Figura 9): SYN (puerto 43952 del cliente → puerto 443 del servidor), SYN, ACK (el paquete mostrado en detalle, con Flags: 0x012), y el ACK final del cliente que completa la conexión. Sobre esa conexión TCP ya establecida corre inmediatamente el handshake TLS 1.3 (Figura 10): primero Client Hello, que incluye el campo SNI (*Server Name Indication*, `data-catalog-mcp-53268160127.northamerica-south1.run.app`), visible en claro pese a ser TLS, ya que el SNI viaja sin cifrar para que el servidor sepa qué certificado presentar, y luego Server Hello, Change Cipher Spec. A partir de ahí toda la comunicación queda cifrada bajo TLS 1.3: sin acceso a las claves de sesión (p.ej. vía `SSLKEYLOGFILE`), Wireshark solo puede mostrar Application Data cifrada (Figura 11), no su contenido. Esto confirma que la capa de transporte (TCP) entrega los datos de forma confiable y ordenada independientemente de que estén cifrados o no: el cifrado es responsabilidad de la capa superior (TLS, entre transporte y aplicación), no de TCP en sí.

Capa de aplicación. Dentro de los registros Application Data de TLS, Wireshark identifica el protocolo interno como HTTP (campo [Application Data Protocol: Hypertext Transfer Protocol] en la Figura 11), aunque no puede decodificar su contenido sin las claves de sesión. Por diseño, el servidor MCP expone un único endpoint HTTP (`POST/`, `Content-Type:application/json`) cuyo cuerpo es, a su vez, un mensaje JSON-RPC. En otras palabras, JSON-RPC actúa como un sub-protocolo de aplicación dentro de HTTP. En la captura local (sin TLS, mismo servidor) este mismo intercambio es perfectamente legible; la Tabla 8 y las figuras referenciadas en §2.1 muestran el contenido real de esos mensajes `initialize`, `tools/list` y `tools/call`.

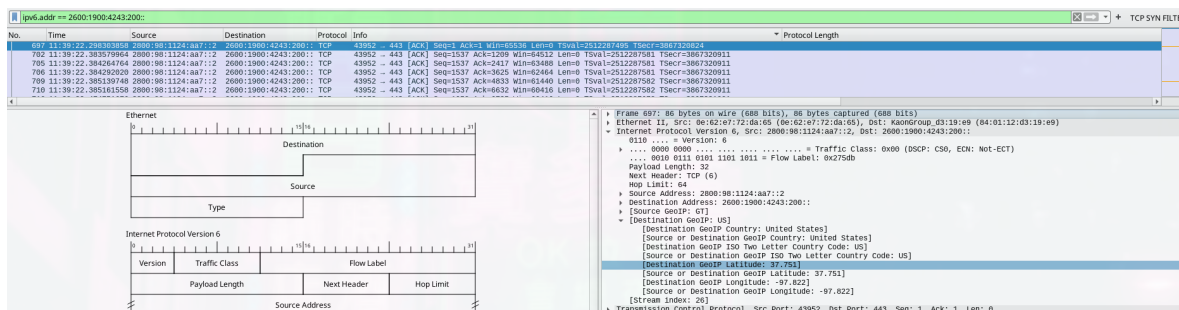


Figura 8: Captura remota: encabezado Ethernet II (capa de enlace) e IPv6 (capa de red) de un paquete ACK del handshake TCP contra Cloud Run.

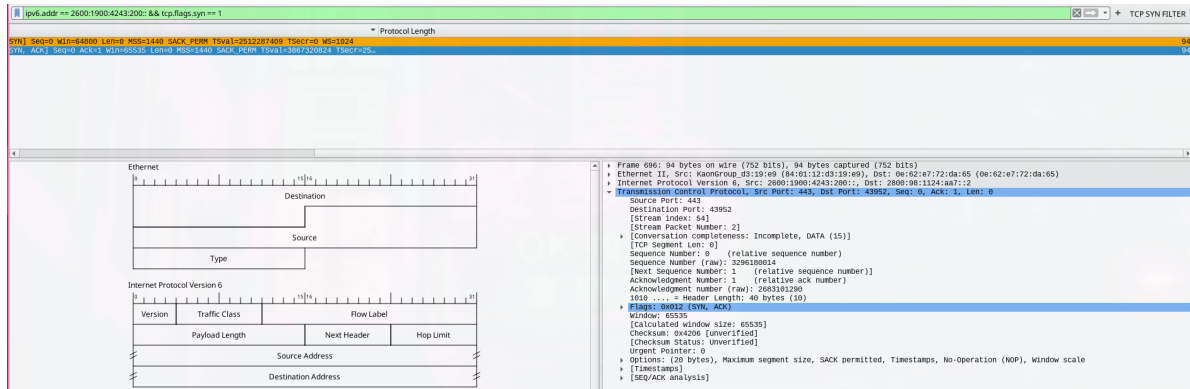


Figura 9: Captura remota: three-way handshake TCP (tcp.flags.syn==1), con el paquete SYN, ACK expandido mostrando puertos y flags.

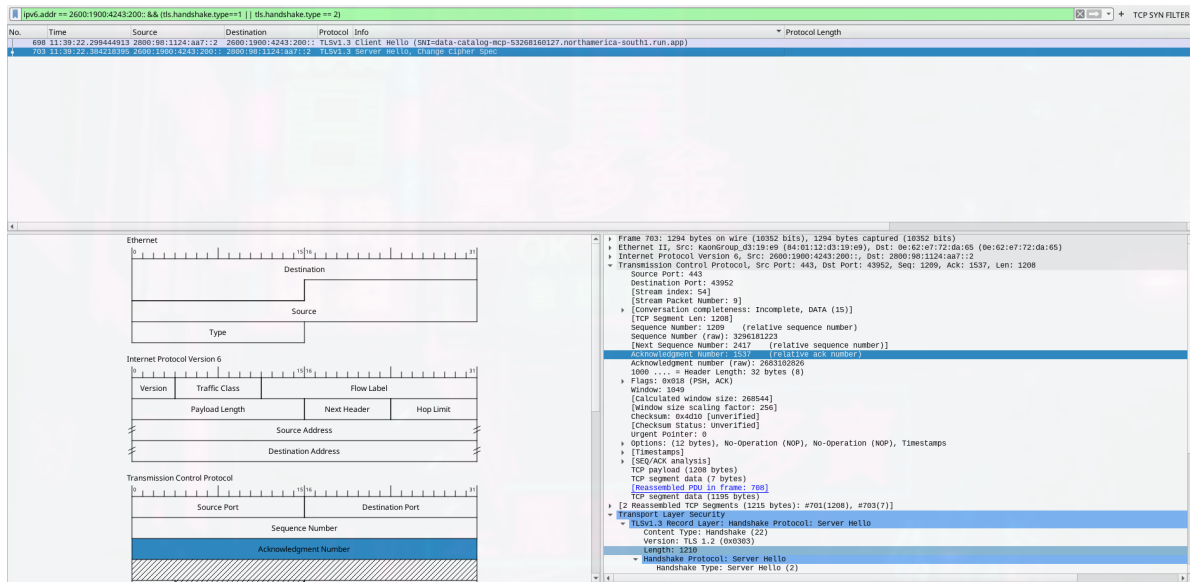


Figura 10: Captura remota: handshake TLS 1.3 (Client Hello con SNI, Server Hello, Change Cipher Spec).

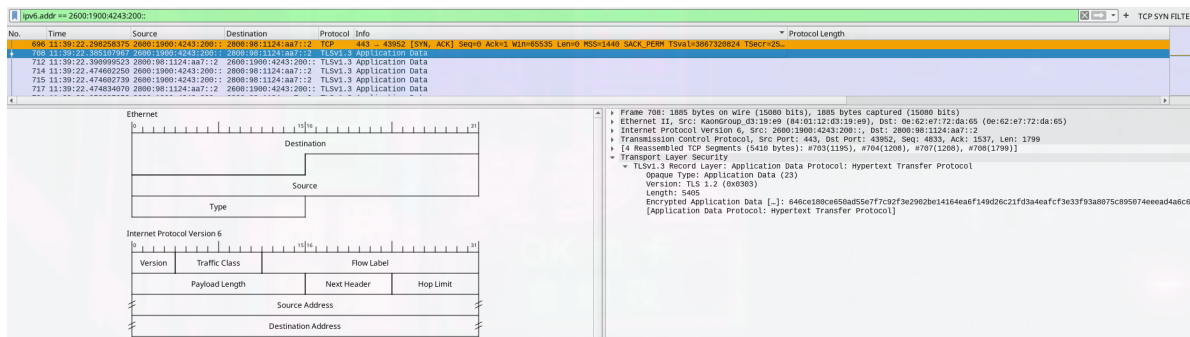


Figura 11: Captura remota: registro TLS de Application Data cifrado, identificado como HTTP a nivel de protocolo pero ilegible sin las claves de sesión.

3. Conclusiones y comentario sobre el proyecto

De las funcionalidades exigidas, todas quedaron implementadas y funcionando de extremo a extremo:

- Conexión al LLM vía su API
- Contexto mantenido entre turnos
- Log visible de las interacciones MCP
- Integración con MCP Filesystem y Git
- Servidor corriendo de manera local como remota
- Análisis de Wireshark

En términos de dificultad, el proyecto no me pareció particularmente complicado a nivel de implementación. El modelo mental de MCP, reducido a lo más esencial es «enviar JSON-RPC, recibir JSON-RPC» y en el fondo eso es lo que se hace en la mayoría de aplicaciones cliente-servidor. Donde sí tuve que prestar mayor atención fue la lectura de la especificación, a diferencia de la mayoría de proyectos que he trabajado no quedaba a mi discreción definir como se lleva a cabo la comunicación o realizar cambios a mi gusto. Aquí cualquier cambio puede hacer que el protocolo deje de ser compatible con algún otro host que lo implemente o que las diferentes partes del proyecto dejen de funcionar en conjunto.

La otra dificultad genuina fue la captura y el análisis con Wireshark, no tanto por el uso de la herramienta en sí sino por conectar correctamente lo que se ve ahí con el resto del curso. Tuve que identificar qué corresponde al *handshake* de sesión de MCP frente al *handshake* TCP/TLS de la capa de transporte, entender por qué la captura remota queda cifrada donde la local no, y relacionar cada capa (enlace, red, transporte, aplicación) con lo que realmente está pasando en el código del proyecto. Es muy diferente poder explicar qué hace el protocolo MCP, a identificar las diferentes capas, que sucede en cada una, etc.

Un momento particular de mayor entendimiento fue al conectar el servidor propio a un *host* completamente ajeno al proyecto (Claude Code) y verlo funcionar sin ningún cambio de código, esto confirma en la práctica que el protocolo realmente logra lo que promete. Pude observar la interoperabilidad entre implementaciones independientes, y no solo entre el cliente y el servidor de este mismo repositorio. En general, el proyecto me dejó un entendimiento bastante más completo de cómo encajan JSON-RPC, HTTP, TCP/IP y TLS entre sí y como se utilizan en el protocolo MCP.

Concluyendo, siento que el proyecto me ayudo a entender de una manera más “integral” los contenidos que hemos estado viendo en el curso.

4. Uso de IA

Para este proyecto se utilizó Inteligencia Artificial (Claude, de Anthropic) en múltiples etapas a lo largo del desarrollo. Principalmente, se le delegaron tareas que involucraran trabajo “manual” y fueran fácilmente verificables por medio de los conocimientos adquiridos desarrollando el proyecto. Por ejemplo:

- Creación de extracto no válido de datos

- Generación de documentación .yaml para los datasets seleccionados
- Implementación de interfaz de tools, al igual que la implementación de los structs completos de cada uno.
- Cambios de API de embeddings o API de LLM, a lo largo del proyecto utilicé Ollama, OpenAI, Gemini y OpenCode. Algunos de ellos son interoperables, mientras que otros no.

También se utilizó para realizar algunas validaciones del protocolo, verificando que se cumplieran los specs 1:1 por si se me había pasado por algo. Por último, el Frontend en su totalidad fue desarrollado utilizando Claude. Personalmente, no me gusta el desarrollo frontend por lo que me dediqué principalmente a afinar algunas cuestiones de HCI. Entre ellas:

- Paleta de colores:
 - Colores neutros / suaves / poco saturados para evitar la sobrecarga cognitiva, además que personalmente me agradan este tipo de colores.
 - Un único color cálido (coral) reservado exclusivamente para elementos interactivos (botones, estado activo de una llamada a herramienta), de forma que signifique “esto se puede presionar, o esto está haciendo algo”.
- Estilo visual “neo-brutalista”: formas rectangulares sin bordes redondeados, contornos marcados, y tipografía monoespaciada. Es una estética asociada a herramientas técnicas / de desarrollador más que a un producto de consumo, la aplicación va dirigida a personas técnicas que trabajan con datos. También preferí mantener la interfaz simple con pocos elementos, comportamiento directo, sin animaciones ni decoración de más.
- Estado del sistema siempre visible: tanto el servidor (`cmd/host`) como el frontend transmiten el estado en tiempo real vía *server-sent events* en vez de un simple indicador de carga. El usuario ve el texto de la respuesta llegando, cuándo se está invocando una herramienta, y el resultado de esa llamada, todo mientras ocurre. Esto corresponde directamente a la heurística de Nielsen de “visibilidad del estado del sistema”.